

BestTime Foot Traffic Integration and Hotness v1 Scoring Pipeline (Madison Demo)

Technical Update for Manager and Engineering Team

Date: 2026-03-01

1. Executive Summary

This document summarizes the Madison demo pipeline that uses BestTime venue foot traffic data to compute crowd and hotness signals. The implementation consists of ``crawldata.py`` for collection/aggregation and ``compute_hotness_v1.py`` for scoring and exports.

- Primary endpoint used for ranking: BestTime `/venues/filter` with geo filtering, `now=true`, and `foot_traffic='both'`.
- Venue-level hotness combines fullness, relative busyness vs daily average, forecast momentum, and a lightweight popularity prior.
- Area-level hotness uses H3 spatial buckets (resolution 9), shrinkage to city mean, and optional neighbor smoothing.
- Final venue score (`venue_hotness_final`) blends venue base hotness and area hotness.
- Attached sample ranked CSV contains 10 venues; attached H3 cell JSON contains 109 cells.

2. Goal and Scope

Goal: create an explainable 'Hot right now' score that is stronger than raw occupancy/fullness alone and can later be combined into a larger hype score. This v1 is demo-oriented and intentionally simple enough to tune.

- Crowd signal = current occupancy-like value from BestTime (`current_busyness`).
- Hotness signal = crowd + relative context + momentum + popularity + spatial context.
- Output is designed for ranking, debugging, and eventual frontend integration.

3. BestTime API Endpoint Strategy

Multiple BestTime options were explored (forecast, live traffic, filter, search, peak hours, surge hours). For the Madison demo, the core operational endpoint is ``/venues/filter``.

3.1 Why `/venues/filter` was selected

- Returns multiple venues per request, which is required for ranking and area aggregation.
- Supports geo filtering (center+radius or bounding box).
- With `foot_traffic='both'`, provides both current and forecast-style busyness arrays.

3.2 Why live traffic is not the main path (Madison)

A test call to the live endpoint for Memorial Union returned an error indicating no live data and no forecast data available at that moment. The pipeline therefore does not rely on live traffic availability for core ranking in Madison.

```
{  
  "status": "Error",
```

```

    "message": "No live data and forecast data available for this venue at this
moment.",
    "analysis": {
        "venue_live_busyness_available": false,
        "venue_forecast_busyness_available": false
    }
}

```

4. Data Collection Pipeline (`crawldata.py`)

`crawldata.py` performs BestTime filter calls, paginates, deduplicates venues, saves raw page JSONs, and writes one aggregated JSON for downstream scoring.

4.1 Setup and configuration

- Reads BESTTIME_API_KEY from .env and raises a clear error if missing.
- Creates output directory besttime_outputs/ if needed.
- Uses requests with timeout=45 and prints API error JSON/text for failed requests.

4.2 Madison geo setup in the script

- Default center+radius mode: CENTER_LAT=43.0731, CENTER_LNG=-89.4012, RADIUS_M=3500.
- Optional bounding box mode is available via USE_BOUNDING_BOX + BBOX config.
- A user category list is stored for local post-filtering/reference (restaurant, bar, cafe, coffee, pizza, brewery, food, nightclub, music venue, concert, theater, entertainment, sports bar, pub, lounge, comedy, arcade, bowling, museum, gallery).

4.3 Filter request parameters (core)

Parameter	Demo setting	Purpose
api_key_private	BESTTIME_API_KEY	Authentication
foot_traffic	both	Return day_raw + day_raw_whole for debugging/scoring
limit	PAGE_SIZE (100)	Pagination page size
page	0..MAX_PAGES-1	Pagination index
lat,lng,radius	43.0731,-89.4012,3500m	Downtown Madison geo filtering
now	true	Align data to current local time
types	Disabled by default	Optional BestTime-side type filter (may mismatch labels)

4.4 Pagination and deduping

- Loop stops when page returns no venues or returned count < PAGE_SIZE.
- Deduping key prefers venue_id; fallback uses name + lat + lng.
- Each raw page is saved as filter_page_<page>_<timestamp>.json for schema inspection.

4.5 Aggregated output schema

The aggregated output written by `crawldata.py` contains:

Top-level key	Meaning
meta	Collection settings (time mode, geo mode, foot traffic mode, pagination, category reference, etc.)
venues_count_unique	Unique venue count after dedupe

venues	Merged unique venue list from all pages
--------	---

In your debugging run, the aggregate structure was confirmed to be ``dict(meta, venues_count_unique, venues)`` and the unique venue count was reported as 411.

5. Raw BestTime Filter Output: Key Fields

A raw filter page response includes a venue list and a ``window`` object. The sample raw file shows a BestTime day window running from Saturday 6AM to Sunday 5AM and includes ``time_local`` + ``time_local_index`` for array indexing.

5.1 Important top-level page fields

Field	Meaning
status	API response status for the filter page
venues	List of venue objects
venues_n	Number of venues returned in the page
window	BestTime time-context object (used for aligning forecast arrays)

5.2 Important ``window`` fields used by scoring

Window field	Meaning	Sample value
time_window_start / time_window_end	Start/end hour of BestTime day window	6 / 5
day_window	Human-readable window text	Saturday 6AM until Sunday 5AM
time_local	Local hour at query time	21 (9PM)
time_local_index	Index to map into day_raw_whole	15

5.3 Important venue fields (raw filter)

Field	Meaning	Used in v1?
venue_id	Venue identifier	Yes
venue_name, venue_address, venue_lat, venue_lng, venue_type	Identity + geo + category	Yes
rating, reviews, price_level	Metadata + popularity prior inputs	rating/reviews yes
day_raw	Current busyness snapshot list (often [x] when now=true)	Yes (preferred)
day_raw_whole	24-point daily busyness profile/forecast	Yes
day_info.day_mean, day_max	Venue daily average / max	day_mean yes
day_info.venue_open_close_v2	Opening hours slots with cross-midnight support	Yes
venue_dwell_time_min/max	Estimated dwell time	No (v1)

6. Scoring Pipeline (``compute_hotness_v1.py``)

``compute_hotness_v1.py`` loads a raw filter file or an aggregated file, detects payload format, extracts venues/window, computes hotness features, optionally computes H3 area hotness, and exports enriched JSON + ranked CSV + H3 cell JSON.

6.1 Input format support and schema drift handling

- Supports raw_filter (status + venues + window).
- Supports bulk (meta + venues), with optional older pages_meta format.

- Supports top-level venue list as a convenience format.
- This compatibility update is important because the aggregated file shape changed during debugging.

6.2 Time extraction and caveat

The script reads ``window.time_local`` and ``window.time_local_index``. If missing, it falls back to a simple default. This is okay for basic runs but can misalign forecast momentum if the aggregate file does not preserve BestTime window context.

- Current busyness is often still correct because ``day_raw[0]`` is preferred when present.
- ``expected_momentum_score`` is the most sensitive field to wrong ``time_local_index``.

6.3 Config and weights (v1)

Variable	Value	Meaning
W_FULLNESS	0.45	Weight for current fullness
W_RELATIVE	0.30	Weight for current vs day mean
W_MOMENTUM	0.15	Weight for forecast momentum
W_POPULARITY	0.10	Weight for popularity prior
W_VENUE_BASE	0.70	Venue base contribution to final score
W_AREA	0.30	Area hotness contribution to final score
H3_RESOLUTION	9	H3 grid size (downtown suitable)
W_CELL_ACTIVITY	0.60	Cell activity weight
W_CELL_ACTIVE_RATIO	0.25	Cell active ratio weight
W_CELL_DENSITY	0.15	Cell density weight
ACTIVE_FULLNESS_THRESHOLD	0.60	Threshold for crowded/active venue in a cell
CELL_DENSITY_N_REF	8	Density normalization reference
CELL_SHRINKAGE_LAMBDA	3.0	Shrinkage toward city mean
W_CELL_SELF / W_CELL_NEIGHBORS	0.70 / 0.30	Neighbor smoothing weights

6.4 Crowd vs Hotness (important product definition)

- ``current_busyness`` is the crowd signal (how full a place is now).
- ``venue_hotness_final`` is the ranking signal for 'Hot right now' mode (crowd + context).
- They are related but different; do not use raw busyness alone as a replacement for hotness.

6.5 Venue-level features and base hotness

The function ``compute_venue_base_hotness(...)`` computes the following:

1. Current busyness from ``day_raw[0]`` if available, else ``day_raw_whole[time_local_index]``.
2. Fullness score = $\text{clip}(\text{current_busyness} / 100)$.
3. Relative score = $\text{clip}((\text{current_busyness} / \max(\text{day_mean}, 1)) / 1.5)$.
4. Expected momentum = positive forecast slope from ``day_raw_whole`` (current vs previous slot), scaled by /40 and clipped.
5. Popularity score = normalized rating $\times$ normalized $\log_{1p}(\text{reviews})$.
6. Open-now gate from ``day_info.venue_open_close_v2['24h']``, including cross-midnight logic.
7. Weighted combination multiplied by open_now.

```
venue_base_hotness =
    open_now * (
        0.45 * fullness_score +
        0.30 * relative_score +
        0.15 * expected_momentum_score +
        0.10 * popularity_score
    )
```

Note: v1 momentum is forecast momentum, not live momentum.

6.6 H3 area hotness (optional, recommended)

If H3 is enabled and importable, each venue is assigned to an H3 cell. Area hotness is then computed from cell-level aggregates and attached back to venues.

8. Assign `h3\_cell` from lat/lng at H3 resolution 9.
9. Group venues by cell.
10. Compute `cell\_activity` (mean venue_base_hotness), `cell\_active\_ratio` (share of fullness ≥ 0.60), and `cell\_density` (normalized n_venues).
11. Combine into `cell\_hotness\_raw` using cell weights.
12. Shrink sparse cells toward city mean using lambda=3.0.
13. Optionally smooth with neighboring H3 cells.

```
cell_hotness_raw =
    0.60 * cell_activity +
    0.25 * cell_active_ratio +
    0.15 * cell_density

cell_hotness_shrunk =
    (n / (n + 3.0)) * cell_hotness_raw +
    (3.0 / (n + 3.0)) * city_mean_cell_hotness

cell_hotness_smoothed =
    0.70 * cell_hotness_shrunk +
    0.30 * neighbor_mean_hotness
```

6.7 Final venue hotness

```
venue_hotness_final =
    clip(0.70 * venue_base_hotness + 0.30 * area_hotness_smoothed)
```

If H3 is disabled/unavailable, the script falls back to `venue\_hotness\_final = venue\_base\_hotness`.

7. Output Files and Usage

Output file	Contents	Use case
hotness_v1_enriched_<ts>.json	Enriched venues + nested hotness_v1 + run metadata	Debugging, backend data inspection
hotness_v1_ranked_<ts>.csv	Flat ranked venue table sorted by venue_hotness_final	Frontend/API ranking, quick QA
hotness_v1_h3_cells_<ts>.json	Per-H3-cell hotness metrics	Heatmap layer, spatial debugging, area context

7.1 Ranked CSV field definitions

Field	Meaning
venue_id	BestTime venue id
venue_name	Venue name
venue_type	Venue type/category from BestTime
venue_lat / venue_lng	Coordinates
rating / reviews / price_level	Metadata; rating/reviews used in popularity prior
h3_cell	H3 cell id (if H3 enabled)
open_now	1=open, 0=closed (open gate applied)
current_busyness	Current busyness (0-100)
day_mean	Average busyness for that day
fullness_score	Normalized current busyness (0-1)
relative_score	Current busyness relative to venue day_mean
expected_momentum_score	Forecast-based momentum proxy
popularity_score	Rating+reviews prior
venue_base_hotness	Venue-only hotness before area context
area_hotness_smoothed	Smoothed H3 area hotness
venue_hotness_final	Final ranking score for hot mode

7.2 H3 cell JSON field definitions

Field	Meaning
n_venues	Number of venues in the cell
cell_activity	Mean venue_base_hotness in the cell
cell_active_ratio	Share of venues with fullness >= threshold
cell_density	Normalized cell density
cell_hotness_raw	Raw weighted cell hotness
city_mean_cell_hotness	Mean raw cell hotness across all cells in the run
cell_hotness_shrunk	Shrinkage-adjusted cell hotness
neighbor_count_used	Number of H3 neighbors included in smoothing
neighbor_mean_hotness	Mean neighbor cell hotness
cell_hotness_smoothed	Final area hotness used for area layer and venue blending

8. Sample Output Snapshot (Attached Files)

- Sample ranked CSV rows: 10
- Open venues in sample ranked CSV: 8/10
- Sample `venue\_hotness\_final` range: 0.103079 to 0.695564
- Sample mean `venue\_hotness\_final`: 0.462127
- H3 cell JSON entries: 109
- City mean cell hotness (from H3 file): 0.143925955896143

8.1 Top 5 venues in the sample ranked CSV

Venue	Type	Open	Busy	DayMean	BaseHot	AreaHot	FinalHot
The Plaza Tavern	BAR	1	80.0	49.0	0.802664	0.445664	0.695564
Eno Vino Downtown	RESTAURANT	1	95.0	79.0	0.750129	0.466172	0.664942
Genna's Lounge	BAR	1	65.0	47.0	0.694926	0.455851	0.623204

Nattspil	RESTAURANT	1	80.0	74.0	0.651447	0.466172	0.595865
Tipsy Cow	RESTAURANT	1	80.0	82.0	0.636133	0.466172	0.585145

8.2 Top H3 cells by smoothed hotness (sample H3 file)

H3 cell	n_venues	activity	active_ratio	density	smoothed_hot
892758008cbffff	37	0.416	0.378	1.000	0.429
892758008c3ffff	10	0.507	0.600	1.000	0.420
89275800853ffff	25	0.414	0.520	1.000	0.416
89275800813ffff	10	0.345	0.600	1.000	0.359
8927580088bffff	25	0.262	0.400	1.000	0.337

9. Worked Example: A Single Ranked Row (The Plaza Tavern)

Field	Value	Why it matters
open_now	1	Closed venues are gated out in venue base hotness
current_busyness	80.0	Raw crowd signal right now
day_mean	49.0	Venue baseline for relative comparison
fullness_score	0.800	Normalized crowd signal (0-1)
relative_score	1.000	How far above/below normal the venue is
expected_momentum_score	0.500	Forecast-based short-term rising signal
popularity_score	0.677	Longer-term popularity prior
venue_base_hotness	0.802664	Venue-only hotness before area context
area_hotness_smoothed	0.445664	Heat of surrounding H3 cell area
venue_hotness_final	0.695564	Final score used for 'Hot right now' ranking

For this row, the final score follows the v1 blend: $\text{final} = 0.70 * \text{venue_base_hotness} + 0.30 * \text{area_hotness_smoothed}$.

10. Runbook (Reproducible Steps)

10.1 Prerequisites

- Python environment with requests and python-dotenv; optional h3 for area scoring.
- A `.env` file containing `BESTTIME_API_KEY`.
- Write access to ``besttime_outputs/``.

10.2 Collect BestTime filter data

```
python3 crawldata.py
```

This saves raw page JSON files and one aggregated file (``all_venues_aggregated_<timestamp>.json``) plus a quick preview in terminal.

10.3 Compute hotness v1

- Set `INPUT\_JSON` in `compute\_hotness\_v1.py` to either a raw filter file or the aggregate file.
- If H3 causes environment issues, set `USE\_H3\_AREA = False` and rerun to get venue-only hotness.

```
python3 compute_hotness_v1.py
```

11. Known Issues and Next Steps

11.1 Known issues / caveats

- BestTime live traffic coverage is inconsistent in Madison; the demo depends on filter endpoint data instead.
- If aggregate files omit BestTime `window` context (`time\_local`, `time\_local\_index`), forecast momentum can be misaligned.
- H3 support depends on local environment/package availability.
- v1 momentum is forecast-based, not live acceleration.
- No category-aware normalization yet (for example bars vs cafes across time-of-day).

11.2 Recommended next improvements

14. Persist BestTime `window` context (or at least `time\_local` and `time\_local\_index`) into aggregate metadata for correct momentum when scoring later.
15. Create mode-specific ranking blends (Best match, Hot right now, Hidden gems, Chill/not crowded) using this hotness and crowd foundation.
16. Add confidence/quality flags for sparse cells, missing hours, or missing busyness profiles.
17. Tune weights with manual QA at multiple times of day before production use.

12. Files Reviewed for This Update

File	Role
crawldata.py	Collection + aggregation
compute_hotness_v1.py	Hotness v1 scoring + exports
filter_madison_small_20260228_213948.json	Raw filter sample (schema and examples)
live_memorial_union_20260228_213949.json	Live endpoint test showing no data available
hotness_v1_ranked_20260301_000938.csv	Ranked venue output sample
hotness_v1_h3_cells_20260301_002627.json	H3 area hotness output sample

End of document. This version is written for both management and engineering review, so it intentionally includes both plain-language explanations and exact scoring formulas.